SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 1

Analytical Problem Solving

COURSE NAME: Deep Learning COURSE CODE: MLA0403 FACULTY : Dr.Arul Raj A.M

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

Assessment Tool Weightage: 35%

Duration: 30 minutes

Marks: 25

Code of Conduct:

I , M Sohail Khan(192525046) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Name of the student: M Sohail Khan M Sohail Khan
QUESTIONS:05 Total Marks:25

Analytical Problem Solving

1. Learning Algorithms (Optimization Behavior)

A machine learning model is trained using gradient descent on a convex loss function. Initially, the learning rate is set very high, and later it is gradually decreased.

Question:
Analyze how the choice of a high initial learning rate followed by decay affects convergence. Under what conditions can this strategy outperform a constant learning rate? Discuss possible risks and justify with reasoning.

Answers:

Effect of a High Initial Learning Rate with Decay

Using a high initial learning rate allows gradient descent to make large updates at the beginning of training. This helps the model reduce the loss quickly when it is far from the optimal solution. As the learning rate gradually decreases, the updates become smaller, allowing the model to converge more smoothly and accurately.

This approach can perform better than using a constant learning rate when the initial value is high enough to speed up learning without causing instability, and the decay is gradual. It is especially effective for smooth convex loss functions because it combines fast early learning with stable convergence near the minimum.

However, there are some risks. If the initial learning rate is too high, the algorithm may overshoot the minimum or even fail to converge. If the learning rate decreases too quickly, learning can slow down before reaching the best solution. On the other hand, if it decays too slowly, the model may continue to oscillate around the minimum.

Overall, a high initial learning rate followed by gradual decay is often more efficient than a constant learning rate because it balances faster learning in the beginning with more precise convergence at the end.

2. Maximum Likelihood Estimation (MLE)

Suppose you are given a dataset generated from a Gaussian distribution with unknown mean μ and known variance σ^2

Question:
Derive the Maximum Likelihood Estimator for μ, and analytically explain how the estimate changes when:

- The dataset contains outliers
- The sample size increases

What does this imply about the robustness of MLE?

Answers:

Maximum Likelihood Estimation (MLE) for the Mean (μ)

For a Gaussian distribution with known variance (σ^2), the likelihood function

is: $L(\mu) = \Pi\, [1 / \sqrt{(2\pi\sigma^2)}] \times \exp[-(xi - \mu)^2 / (2\sigma^2)]$

Taking the natural log of the likelihood:

$\log L(\mu) = \text{Constant} - (1 / 2\sigma^2) \times \Sigma(xi - \mu)^2$

Differentiate with respect to μ and set it equal to zero:

$d(\log L)/d\mu = (1 / \sigma^2) \times \Sigma(xi - \mu) = 0$

Solving for μ:

$\Sigma xi - n\mu = 0$

$\hat{\mu} = (\Sigma xi) / n$

Therefore, the Maximum Likelihood Estimator (MLE) of the mean is simply the sample mean.

Effect of Outliers:
Outliers can greatly affect the estimate because the sample mean is sensitive to extreme values. A few unusually large or small observations can shift the estimated mean away from the true value.

Effect of Increasing Sample Size:
As the sample size increases, the estimate becomes more stable and gets closer to the true mean. The effect of random variation becomes smaller, making the MLE more reliable.

Implication for Robustness:
MLE works very well when the data follows a normal distribution, but it is not robust to outliers because it depends on the sample mean. Extreme values can have a strong influence on the final estimate.

3. Building a Machine Learning Algorithm (Model Selection)

You are designing a classification algorithm for a dataset with 10,000 features but only 500 training samples.

Question:
Analyze the challenges involved in training such a model. Propose a strategy (algorithm pipeline) to handle this situation and justify each step (e.g., feature selection, regularization, dimensionality reduction). Explain how your approach mitigates overfitting.

Answers:

Building a Machine Learning Algorithm (Model Selection)

When a dataset has 10,000 features but only 500 training samples, the main challenge is overfitting. The model may learn noise instead of meaningful patterns because there are far more features than training examples. Training also becomes slower and more computationally expensive, and many features may be irrelevant or redundant.

Proposed Algorithm Pipeline:

1. Data Preprocessing: Clean the data, handle missing values, and normalize or standardize the features to improve model performance.
2. Feature Selection: Remove irrelevant or less important features using methods such as correlation analysis or L1 (Lasso) feature selection. This reduces the number of input features and keeps only the most useful ones.
3. Dimensionality Reduction: Apply Principal Component Analysis (PCA) to reduce the feature space while preserving most of the important information. This makes the model simpler and faster.

4. Regularization: Train a classifier with L1 or L2 regularization (such as Logistic Regression or a Linear SVM). Regularization penalizes large model weights, reducing model complexity. 5. Model Validation: Use k-fold cross-validation to evaluate the model and tune hyperparameters, ensuring it generalizes well to unseen data.

How This Reduces Overfitting:

- Feature selection removes unnecessary features that may introduce noise.
- Dimensionality reduction simplifies the dataset while keeping the most important information. • Regularization prevents the model from fitting the training data too closely. • Cross-validation helps choose a model that performs well on new data rather than just the training set.

Overall, this pipeline creates a simpler, more generalizable model that performs better on unseen data and reduces the risk of overfitting.

4. Neural Networks (Multilayer Perceptron - MLP)

Consider a Multilayer Perceptron with one hidden layer using sigmoid activation functions.

Question:
Explain analytically why adding more hidden neurons increases the representational power of the network. Then, reason why this does not always lead to better performance. Support your answer using concepts like overfitting and generalization.

Answers:

Neural Networks (Multilayer Perceptron - MLP)

A Multilayer Perceptron (MLP) with one hidden layer and sigmoid activation functions can learn complex, non-linear relationships between inputs and outputs. Adding more hidden neurons increases the network's representational power because each neuron learns a different feature or pattern from the data. With more neurons, the network can model more complex decision boundaries and capture finer details in the dataset.

However, adding more hidden neurons does not always improve performance. A larger network has more parameters, making it easier to memorize the training data instead of learning general patterns. This leads to overfitting, where the model performs very well on the training data but poorly on new, unseen data.

A smaller or properly regularized network usually has better generalization, meaning it can make accurate predictions on unseen examples. Techniques such as dropout, regularization, and early stopping help reduce overfitting and improve generalization.

In conclusion, increasing the number of hidden neurons improves the MLP's ability to learn complex patterns, but too many neurons can cause overfitting. The best performance is achieved by choosing a network size that balances learning capacity with good generalization.

5. Backpropagation, SGD & Curse of Dimensionality

A deep neural network is trained using Stochastic Gradient Descent (SGD) on a very high-dimensional dataset.

Question:
Analyze how the **curse of dimensionality** impacts:

- Gradient updates in backpropagation
- Convergence speed of SGD
- Model generalization

Propose at least two techniques to overcome these issues and justify them

analytically. **Answers:**

Backpropagation, SGD & Curse of Dimensionality

When training a deep neural network on a high-dimensional dataset, the curse of dimensionality makes learning more difficult because the number of features is very large compared to the available data.

Impact on Gradient Updates (Backpropagation):
With many features, the network has a large number of weights to update. This makes the gradients noisy and less informative, making it harder for backpropagation to find the best direction for updating the weights.

Impact on SGD Convergence:
Stochastic Gradient Descent (SGD) may converge more slowly because it has to optimize a much larger parameter space. It often requires more iterations and careful tuning of the learning rate to reach a good solution.

Impact on Model Generalization:
High-dimensional data increases the risk of overfitting. The model may memorize the training data instead of learning meaningful patterns, resulting in poor performance on unseen data.

Techniques to Overcome These Issues:

1. Feature Selection or Dimensionality Reduction (PCA):
   Remove irrelevant features or reduce the number of dimensions while keeping the most important information. This reduces model complexity, speeds up training, and lowers the risk of overfitting.
2. Regularization (L1/L2 or Dropout):
   Regularization limits the complexity of the model by penalizing large weights, while dropout randomly deactivates neurons during training. Both techniques prevent the network from relying too much on specific features and improve generalization.

Conclusion:
The curse of dimensionality makes gradient updates less effective, slows down SGD convergence, and increases overfitting. Using dimensionality reduction and regularization helps simplify the model, improves training efficiency, and allows the network to generalize better to new data.